System Verification and Validation Plan for Project Proxi

Team #23, Project Proxi
Savinay Chhabra
Amanbeer Singh Minhas
Gourob Podder
Ajay Singh Grewal

April 6, 2026

Revision History

Date	Ver.	Notes
30/10/2025	1.0	Added initial V&V plan.
08/03/2026	1.1	Added introductory text for Section 1. Corrected spelling, grammar, and document flow issues. Replaced magic numbers with symbolic constants where appropriate. Standardized terminology and stylization (for example, macOS). Moved table titles above tables and added in-text references. Added a role reassignment plan for V&V team members. Refined review checklists so items are atomic and directly checkable. Clarified specific WCAG checks used for accessibility verification. Added tests for unintended and unauthorized actions, including destructive and replay-style scenarios. Included draft checklists referenced by the plan. Clarified traceability with requirement-to-test matrix coverage. Made walkthroughs, inspections, stakeholder reviews, and project-specific extras more explicit and justified.

Contents

1 General Information **1**
- 1.1 Summary . . . 1
- 1.2 Objectives . . . 1
- 1.3 Challenge Level and Extras . . . 2
- 1.4 Relevant Documentation . . . 3

2 Plan **3**
- 2.1 Verification and Validation Team . . . 4
- 2.2 Role Reassignment and Contingency Plan . . . 4
- 2.3 SRS Verification . . . 5
- 2.4 Design Verification . . . 7
- 2.5 Verification and Validation Plan Verification . . . 8
- 2.6 Implementation Verification . . . 10
- 2.7 Automated Testing and Verification Tools . . . 11
- 2.8 Software Validation . . . 12

3 System Tests **13**
- 3.1 Tests for Functional Requirements . . . 13
  - 3.1.1 Input and Recognition Requirements . . . 14
  - 3.1.2 Intent Understanding and Task Execution . . . 16
  - 3.1.3 Feedback and Responsiveness . . . 17
  - 3.1.4 Contextual Memory and Logging . . . 18
  - 3.1.5 Usability and Safety . . . 19
- 3.2 Tests for Non-Functional Requirements . . . 21
  - 3.2.1 Look and Feel Requirements . . . 21
  - 3.2.2 Usability and Humanity Requirements . . . 22
  - 3.2.3 Performance and Reliability Requirements . . . 23
  - 3.2.4 Operational and Environmental Requirements . . . 25
  - 3.2.5 Maintainability and Support Requirements . . . 26
  - 3.2.6 Security Requirements . . . 27
  - 3.2.7 Cultural Requirements . . . 28
  - 3.2.8 Compliance Requirements . . . 28
- 3.3 Traceability Between Test Cases and Requirements . . . 29

4 Unit Test Description **32**
4.1 Unit Testing Scope 32
4.2 Tests for Functional Requirements 32
4.2.1 Tests for BH-Input 32
4.2.2 Tests for BH-NLU 34
4.2.3 Tests for BH-Plan 35
4.2.4 Tests for BH-Safety 36
4.2.5 Tests for BH-Feedback 38
4.2.6 Tests for BH-UI 39
4.3 Tests for Non-Functional Requirements 40
4.3.1 Tests for BH-Feedback 41
4.3.2 Tests for BH-Safety 41

5 Appendix **43**
5.1 Symbolic Parameters 43
5.2 Usability Survey and Acceptance Criteria 45

This document outlines the Verification and Validation plan for Project Proxi. It explains how our team plans to evaluate the system we are building against its requirements. The plan describes what tests will be run, how documents and designs will be reviewed and validated, and how the evidence collected traces back to the original SRS document. The main goal is to make sure Project Proxi is reliable, easy to use, and genuinely helpful for the people who rely on it for accessibility and productivity.

1 General Information

1.1 Summary

The aim of this document is to present the verification and validation plan for Project Proxi. Project Proxi is an AI-powered desktop assistant that allows its users to interact with their computer using voice and text. The target users for Project Proxi include seniors, individuals with disabilities, and power users who want a more accessible, efficient, and convenient way to perform computer tasks.

1.2 Objectives

The main objective of the Verification and Validation (V&V) plan is to confirm that Project Proxi satisfies its stated functional and non-functional requirements and fulfills its primary purpose of improving computer accessibility and efficiency for seniors, individuals with disabilities, and power users. The principal objectives that this V&V plan aims to meet are:

- **Verify Functional Correctness:** Ensure that the features described in the SRS are implemented and behave as expected through unit, integration, and system-level testing.
- **Validate Usability and Accessibility:** Demonstrate that users within the target demographic can effectively operate the system using voice and text while meeting the relevant accessibility requirements, including WCAG 2.2 guidance for accessible user interfaces.
- **Assess Performance and Reliability:** Confirm that the application meets performance, stability, and error-handling goals as defined in the SRS document.

- **Hardware Compatibility Testing:** Meet verification goals on a variety of consumer hardware devices. Specialized or adaptive devices will not be included; only commercially available Windows and macOS computers.

Out-of-scope objectives include extensive long-term field usability studies, comprehensive privacy compliance, and specialized hardware testing. The project will assume that any external libraries have already been verified by their respective development teams. These exclusions are justified given the project's limited time, budget, and academic context. Verification efforts are therefore directed toward objectives that help meet core software requirements.

1.3 Challenge Level and Extras

This project does not include a designated challenge level, as its primary focus is on the development and validation of a functional, accessible, and user-friendly AI-powered desktop assistant. The project scope aligns with the general expectations of the capstone course and emphasizes reliability, usability, and compliance with accessibility standards rather than advanced AI research or complex algorithmic innovation.
This project includes the following approved extras:

- **User Manual:** A user manual will document installation, setup, and everyday usage of the software through clear, step-by-step instructions. This extra is important because Project Proxi is intended for seniors, individuals with disabilities, and power users with varying levels of technical proficiency. The manual supports onboarding and independent use by explaining core workflows, common commands, safety confirmations, and troubleshooting guidance in a structured and easy-to-reference format.

- **Usability Report:** A usability report will be developed to assess the system's core objectives and requirements. It will summarize the results of testing as specified in the SRS document. This extra is important because usability and accessibility are central goals of Project Proxi, and the report will help identify issues that can be used to improve the system before final delivery.

1.4 Relevant Documentation

The following documentation is directly relevant to the verification and validation activities for Project Proxi:

- **Development Plan [Chhabra et al. (2025a)]:** This document defines the overall project workflow, timelines, and milestones for the different phases of the project.
- **Problem Statement and Goals [Chhabra et al. (2025c)]:** This document defines the problems, goals, and stakeholders of the project.
- **Software Requirements Specification [Chhabra et al. (2025d)]:** Describes the functional and non-functional requirements of the project.
- **Hazard Analysis [Chhabra et al. (2025b)]:** This document identifies any potential sources of harm or risk that may be associated with the project. Potential risks such as unauthorized access, misuse, or data breaches and their mitigation strategies are documented here.
- **Usability Report [Chhabra et al. (2025e)]:** Includes results of user evaluation and beta testing during the validation stage. It summarizes key findings related to the system meeting its design and usability goals.
- **User Manual [Chhabra et al. (2025f)]:** Documents how to install, configure, and use the software, including command examples, core workflows, and troubleshooting steps to support users with varying technical proficiency.

2 Plan

This section outlines the strategy for Verification and Validation (V&V) across all phases of the Project Proxi lifecycle. The plan defines the roles and responsibilities of the V&V team and specifies structured approaches for verifying the Software Requirements Specification (SRS), Design, and Implementation, as well as the approach for Software Validation. This systematic process increases confidence in the correctness, accessibility, and reliability of the final system.

Verification ensures that the system is built correctly and meets its stated requirements, while validation confirms that the right system has been built

for its intended users. Both static techniques (reviews, inspections, and walkthroughs) and dynamic techniques (testing and user evaluations) are used. All artifacts, results, and issues are tracked through GitHub for full traceability between requirements, test cases, and evidence.

2.1 Verification and Validation Team

The Verification and Validation team for Project Proxi is the Proxi project team itself. Since the team is small, all members contribute to V&V activities in addition to their development responsibilities. Table 1 summarizes the primary V&V responsibilities assigned to each team member.

All verification tasks and results are peer reviewed, documented, and tracked in GitHub to maintain accountability and consistent quality assurance.

Table 1: Verification and Validation Team Roles

Name	**Role**	**V&V Responsibilities**
Savinay Chhabra	Team Lead / Project Manager	Coordinates all V&V activities, schedules document and design reviews, ensures traceability matrices are complete, and validates plan feasibility.
Amanbeer Singh Minhas	Verification Engineer / Documentation Lead	Authors test specifications, manages accessibility validation (WCAG 2.2), executes functional and non-functional tests, and maintains V&V evidence and metrics.
Gourob Podder	AI / Backend Developer	Verifies the accuracy and latency of voice, text, and natural language processing modules, conducts integration and performance tests, and performs code inspections.
Ajay Singh Grewal	Front-End / Integration Engineer	Implements automated UI tests, validates responsiveness and error handling, and ensures cross-platform consistency and workflow reliability.

2.2 Role Reassignment and Contingency Plan

If timelines, availability, or project risks change, V&V responsibilities will be rebalanced using the following procedure:

- The Team Lead identifies blocked work items and proposes reassignment within one business day.
- At least one backup reviewer is maintained for each critical activity, including SRS review, design review, test execution, and evidence tracking.
- Reassignments are recorded in a GitHub issue labeled `vnv-role-change`, including the new owner, due date, and rationale.
- The traceability matrix and meeting notes are updated in the same sprint to reflect responsibility changes.

2.3 SRS Verification

The Software Requirements Specification (SRS) will be verified using a structured review process involving internal checks, peer feedback, and consistency checks. The goal is to confirm that all requirements are clear, consistent, complete, and ready for design and testing.

Planned Verification Stages:

- **Internal Team Review:** Each team member reviews assigned SRS sections to ensure requirements are clearly written, uniquely numbered, feasible, and testable. Feedback and questions are added as GitHub issues under the `SRS-review` tag for tracking.
- **Peer Team Feedback:** Another project team will review the SRS to identify unclear, ambiguous, inconsistent, or missing requirements. Their comments will be discussed in a short review meeting, and agreed changes will be included in the next SRS revision.
- **Consistency and Traceability Check:** Manual and automated checks will confirm that every requirement label (functional, non-functional, and constraint-related) appears in the traceability matrix and links to one or more planned verification activities in the V&V Plan. Any missing, duplicate, or inconsistent identifiers will be flagged for correction.

- **Team Validation Meeting:** After all updates, the team will meet to confirm that project goals such as accessibility, reliability, safety, and usability are fully captured in the SRS. The final version will then be approved as the baseline for design and testing.

SRS Review Checklist:

- Each requirement has an identifier.
- Each identifier is unique.
- Each requirement contains one main verifiable idea.
- Each requirement uses direct language.
- Undefined terms are avoided or added to the glossary.
- Each requirement states observable behaviour or a measurable outcome.
- Each requirement is feasible within capstone scope.
- Each requirement can be verified by inspection, analysis, or test.
- Each requirement is consistent with the problem statement.
- Each requirement is consistent with the project goals.
- Each safety-related requirement is consistent with the hazard analysis.
- Accessibility requirements reference WCAG 2.2 where applicable.
- Voice interaction requirements are represented where needed.
- Text interaction requirements are represented where needed.
- High-risk actions include confirmation or mitigation requirements.
- Each requirement appears in the traceability matrix.

2.4 Design Verification

The design of Project Proxi will be verified through focused peer reviews, walkthroughs, and checklists to confirm that the architecture, interfaces, and data flows align with the approved SRS. Each review will help identify inconsistencies or missing details before implementation begins.

Planned Verification Activities:

- **Internal Design Review:** Team members will examine assigned modules to ensure the design is modular, consistent with SRS requirements, and feasible for implementation. Each reviewer will complete a checklist covering module responsibilities, interface definitions, traceability, and error handling.
- **Cross-Team Design Review:** Another project team will review the design documents and provide feedback on readability, logic, completeness, and consistency. Their comments will be logged as GitHub issues tagged `Design-review` and resolved in the next revision.
- **Walkthrough Meeting:** The team will conduct a collaborative walkthrough in which each member explains the purpose, inputs, outputs, and dependencies of their assigned subsystem. Meeting notes will record identified issues, action items, and owners.
- **Traceability Check:** A traceability matrix will confirm that every major design component maps to at least one SRS requirement and that every SRS requirement is supported by one or more design elements. Any unmatched modules or requirements will be flagged for revision before coding starts.

Design Review Checklist Items:

- Each module has a unique name.
- Each module states its main purpose.
- Each module lists its responsibilities.
- Each module lists its inputs.
- Each module lists its outputs.

- Each interface names the caller module.
- Each interface names the callee module.
- Each interface lists exchanged data items.
- Data item names match across connected modules.
- Data item meaning is consistent across connected modules.
- Each major data flow identifies its source.
- Each major data flow identifies its destination.
- Each major data flow includes the expected transformation or decision.
- Error handling is identified where failure is possible.
- Overlapping module responsibilities are avoided.
- Accessibility-related design elements are identified explicitly.
- Voice interaction support appears in the design.
- Text interaction support appears in the design.
- Each major component traces to at least one SRS requirement.

2.5 Verification and Validation Plan Verification

The Verification and Validation (V&V) Plan itself will also be verified to ensure it is complete, consistent, and realistic. This review confirms that the plan defines clear responsibilities, test coverage, and measurable evidence of software quality.

Planned Verification Activities:

- **Internal Review:** Team members will inspect the V&V Plan for completeness, logical flow, and consistency with other project documents such as the SRS, MIS, MG, and Hazard Analysis. Issues will be logged on GitHub under the tag `VnV-review`.

- **Peer Team Review:** A partner project team will review the plan to provide external feedback on clarity, organization, feasibility, and coverage. Comments will be discussed in a team meeting, and accepted suggestions will be incorporated into the next version.
- **Cross-Document Consistency Check:** The team will confirm that all V&V activities listed here are consistent with the SRS, MIS, MG, and Hazard Analysis and that requirement identifiers match across artifacts.
- **Review Process Validation:** Small intentional edits, such as removing a trace link or altering a test identifier, will be introduced to confirm that the team's review process can detect inconsistencies. This activity validates the effectiveness of the review workflow.

V&V Plan Review Checklist:

- Each cited document title is correct.
- Each cited document identifier is correct.
- Roles and responsibilities match Table 1.
- Each major section includes a short purpose statement.
- Each planned activity identifies expected evidence.
- Each test identifier is unique.
- Each test identifier is used consistently across the document.
- Internal review steps are stated explicitly.
- Peer review steps are stated explicitly.
- The plan fits the project timeline and tool availability.

2.6 Implementation Verification

The implementation of Project Proxi will be verified through a combination of dynamic and static techniques to ensure the system performs correctly, reliably, and according to its specified design. This section summarizes how unit, integration, and system testing activities will be supported by automated tools and structured reviews.

Dynamic Verification:

- **Unit Testing:** Each software module will have dedicated unit tests as outlined in the unit testing plan. Tests will confirm correct outputs, data flow, state changes, and error handling for input processing, planning, execution, and UI modules. Coverage results will be monitored through automated reports to ensure that major functions are exercised.

- **Integration and System Testing:** Combined module testing will validate interactions between the input pipeline, user interface, planning components, execution components, and backend services. These tests correspond directly to the functional and non-functional test cases listed in this document.

- **Mutation Testing:** To evaluate the quality of the test suite, small artificial faults (mutants) will be introduced into the codebase. Existing unit and system tests will be rerun to confirm that they detect the injected faults. A high mutation score will increase confidence in the strength and completeness of the test suite.

Static Verification:

- **Code Walkthroughs and Inspections:** Code walkthroughs will occur before major feature merges or at least once per sprint for active modules. The author will present the change, linked design elements, linked test cases, and expected behaviour. Review notes, blocking issues, and follow-up actions will be recorded in pull request comments or GitHub issues. A walkthrough is considered complete only when all blocking issues have an owner and a resolution plan.

- **Static Analysis Tools:** Linters and analyzers, such as ESLint, Pylint, or similar tools, will be used to detect common issues, unused variables, inconsistent style, and potential logic errors. Results will be documented as part of the verification evidence.

- **Code Review via Pull Requests:** All implementation changes will go through GitHub pull requests with at least one reviewer. Reviewers will verify coding standards, style, alignment with design diagrams, and whether the change preserves intended behaviour without introducing unintended actions.

2.7 Automated Testing and Verification Tools

Automated tools will be used throughout the implementation phase to support testing, continuous integration, and quality assurance. These tools help reduce manual effort, detect regressions early, and provide measurable evidence of software reliability and maintainability.

Automated Testing Framework:

- The project will use `pytest` for automated unit, integration, and scripted scenario tests. Each major module will have dedicated test files and fixtures to verify inputs, outputs, state changes, and edge cases.

- Scripted scenario tests will simulate realistic user interactions by submitting selected voice and text commands, capturing system responses, and comparing them against expected results.

- For repeatable testing, scenario tests will use a predefined command set maintained by the team. The full set of commands used for automated testing will be listed explicitly in the corresponding test cases or in an appendix, rather than described only as a sample count.

Continuous Integration and Delivery (CI/CD):

- GitHub Actions will be configured to automatically run linting and automated tests whenever code is pushed or a pull request is opened. The pipeline will help detect regressions early and ensure that changes are reviewed against a consistent verification process before merge.

Static Analysis and Linters:

- Static analysis tools such as `pylint` and `flake8` will be used to detect syntax errors, unused variables, inconsistent style, and maintainability issues. Results will be logged and tracked as part of the verification evidence.

Build and Automation Utilities:

- The project will use `make` commands and supporting scripts to automate environment setup, dependency installation, linting, and test execution. This helps ensure consistent verification results across development machines and the CI/CD pipeline.

2.8 Software Validation

Software validation confirms that Project Proxi satisfies user needs and performs as intended in realistic scenarios. The focus is on showing that the system achieves its goals of accessibility, responsiveness, safety, and ease of use for seniors, individuals with disabilities, and power users.

Planned Validation Activities:

- **User Testing Sessions:** A small group of peers and volunteers will perform task-based evaluations, such as launching applications, entering text, and managing files through voice and text interaction. Their feedback will be used to assess ease of learning, efficiency, accessibility, and overall satisfaction.

- **Accessibility Validation:** Validation will confirm that the interface supports the accessibility goals defined in the SRS and aligns with relevant WCAG 2.2 guidance. The evaluation will explicitly check WCAG 2.2 Success Criteria 1.4.3 (Contrast (Minimum)), 1.4.4 (Resize Text), 1.3.1 (Info and Relationships), and 4.1.2 (Name, Role, Value) where applicable to the final interface.

- **Stakeholder Review Session:** The team will conduct one structured review session as a validation milestone. The participants will include course staff, classmates, and peer reviewers acting as informed external stakeholders. The session will include a short system overview, a

guided demonstration of core tasks, a demonstration of one high-risk confirmation flow, and a brief feedback period. The core demonstration scenarios will include email summarization, calendar event creation, browser search and summary, Notion page creation, and a privileged action that requires explicit confirmation. Feedback will be collected through a short review form and meeting notes. Any issue reported by at least two reviewers, or any issue that blocks a core task, will be logged as a GitHub issue and considered for revision before final delivery.

- **Automated Scenario Validation:** Scripted scenario tests will submit a predefined and fully enumerated set of commands to the system, record the observed outputs, and compare them with expected behaviour. These tests will help validate recurring tasks, command interpretation accuracy, response consistency, and protection against unintended actions.

- **External Data and Comparison:** Where appropriate, publicly available speech or language datasets may be used to compare command interpretation behaviour against baseline expectations. This provides an additional reference point for judging consistency and robustness.

3 System Tests

Each test is specific and measurable, providing sufficient detail for replication once the system is built. The tests are grouped into logical areas covering input handling, natural language understanding, task execution, feedback mechanisms, memory management, logging, and system safety.

Unless otherwise stated, repeated thresholds and sample sizes reference the symbolic parameters defined in Appendix A. These thresholds are chosen to align with SRS fit criteria and the realistic limits of a capstone project, rather than assuming perfect behaviour from speech- and language-based features.

3.1 Tests for Functional Requirements

The following tests verify that each functional requirement defined in the SRS has been properly implemented and behaves as intended. Each test

specifies the type of testing, initial system state, inputs, expected results, and how the test will be performed.

3.1.1 Input and Recognition Requirements

1. **Test FUNC.R.1 – Speech and Text Input Acceptance**

   Type: Functional / Manual Testing

   Initial State: System initialized and idle at the main interface.

   Input/Condition: Users provide the following `CMD_SET_SIZE` valid commands. Commands 1–10 are entered by text, and commands 11–20 are spoken through the microphone interface.

   1. Summarize my latest unread email.
   2. Draft an email to John about tomorrow's team meeting.
   3. Show my calendar events for tomorrow.
   4. Create a calendar event for Friday at 2 PM called Team Sync.
   5. Search the web for Hamilton weather and summarize the result.
   6. Search the web for nearby walk-in clinics.
   7. Make a Notion page called Drinking Water Tracker.
   8. Add "Drink 8 glasses today" to my Drinking Water Tracker page.
   9. Summarize the file named Capstone Notes.
   10. What can I say?
   11. Summarize my latest unread email.
   12. Draft an email to John about tomorrow's team meeting.
   13. Show my calendar events for tomorrow.
   14. Create a calendar event for Friday at 2 PM called Team Sync.
   15. Search the web for Hamilton weather and summarize the result.
   16. Search the web for nearby walk-in clinics.
   17. Make a Notion page called Drinking Water Tracker.
   18. Add "Drink 8 glasses today" to my Drinking Water Tracker page.
   19. Summarize the file named Capstone Notes.

20. What can I say?

Output/Result: The system correctly accepts and interprets at least `CMD_ACCEPT_MIN` of the valid inputs, and each accepted input is converted into the expected internal command representation and displayed result.

How test will be performed: Each input and resulting interpreted command will be logged automatically. The test supervisor will compare the interpreted result against the expected result for the full command set and calculate the success rate. Any rejected, misinterpreted, or malformed results will be recorded for analysis.

Functional Requirements Covered: FUNC.R.1

2. **Test FUNC.R.2 – Speech-to-Text Accuracy**

   Type: Functional / Automated Evaluation

   Initial State: System with speech-to-text enabled and configured in a controlled acoustic environment.

   Input/Condition: The following `STT_CMD_COUNT` spoken commands are each recorded from `STT_SPEAKER_COUNT` speakers:

   1. Summarize my latest unread email.
   2. Draft an email to John about tomorrow's team meeting.
   3. Show my calendar events for tomorrow.
   4. Create a calendar event for Friday at 2 PM called Team Sync.
   5. Search the web for Hamilton weather and summarize the result.
   6. Search the web for nearby walk-in clinics.
   7. Make a Notion page called Drinking Water Tracker.
   8. Add "Drink 8 glasses today" to my Drinking Water Tracker page.
   9. Summarize the file named Capstone Notes.
   10. What can I say?

   Output/Result: The resulting transcriptions achieve at least `STT_ACC_MIN` word accuracy across the full sample set.

How test will be performed: The recorded audio samples will be batch fed into the speech-to-text component. Output transcripts will be compared against reference transcripts using a word error rate metric, and the overall accuracy will be computed from the full set.

Functional Requirements Covered: FUNC.R.2

3.1.2 Intent Understanding and Task Execution

1. **Test FUNC.R.3 – Intent Recognition Accuracy**

   Type: Functional / Automated Testing

   Initial State: Natural language understanding component initialized with the current intent parsing configuration.

   Input/Condition: The system is given `INTENT_SAMPLE_COUNT` labeled inputs covering the supported intents. The supported intents are: summarize email, draft email, show calendar, create calendar event, browser search, summarize browser result, create Notion page, update Notion page, summarize file, and show help or examples.

   Output/Result: At least `INTENT_ACC_MIN` of the labeled inputs are assigned the correct ground-truth intent.

   How test will be performed: A test script will feed the labeled inputs into the intent parser. Predicted intents will be logged and compared against human-labeled references to compute intent-level accuracy.

   Functional Requirements Covered: FUNC.R.3

2. **Test FUNC.R.4 – Task Planning and Execution Success Rate**

   Type: Functional / Integration Testing

   Initial State: System active with required tools, permissions, and network connectivity available.

   Input/Condition: The following representative high-level commands are executed sequentially:

   1. Summarize my latest unread email.
   2. Draft an email to Sarah about the capstone update.
   3. Draft an email to John saying I will join in 10 minutes.

4. Show my calendar events for tomorrow.
5. Create a calendar event for Friday at 2 PM called Team Sync.
6. Move my Team Sync event to 3 PM.
7. Search the web for Hamilton weather.
8. Summarize the current browser result.
9. Search the web for nearby pharmacies.
10. Summarize the top result in two sentences.
11. Make a Notion page called Drinking Water Tracker.
12. Add "Drink 8 glasses today" to my Drinking Water Tracker page.
13. Make a Notion page called Capstone Tasks.
14. Add "Finish V&V plan" to my Capstone Tasks page.
15. Summarize the file named Capstone Notes.
16. Read the first paragraph of Capstone Notes aloud.
17. Show the summary again.
18. What can I say?
19. Undo the last action.
20. Cancel the current action.

Output/Result: At least `TASK_SUCCESS_MIN` of the commands are completed successfully and produce the intended task outcome in the interface.

How test will be performed: Commands will be executed one at a time. Testers will verify whether each displayed result, draft, summary, or state change matches the intended user goal. Failures will be logged along with their cause, such as planning failure, permission failure, or execution failure.

Functional Requirements Covered: FUNC.R.4

3.1.3 Feedback and Responsiveness

1. **Test FUNC.R.5 – Response Feedback Timing**

   Type: Functional / Manual Timing

Initial State: System ready to execute user commands.

Input/Condition: The following common tasks are executed: summarize latest unread email, draft an email, show tomorrow's calendar, create a calendar event, search the web for Hamilton weather, summarize the current result, search for nearby pharmacies, make a Notion page, summarize a file, and show help commands.

Output/Result: Each command generates visible or spoken feedback within `FEEDBACK_DELAY_MAX` after task completion or after the system determines that the task cannot be completed.

How test will be performed: A timer will start at the moment the task finishes or the failure state is known. Feedback appearance time will be recorded for each trial, and all observed times will be compared against the threshold.

Functional Requirements Covered: FUNC.R.5

3.1.4 Contextual Memory and Logging

1. **Test FUNC.R.6 – Contextual Follow-Up Interpretation**

   Type: Functional / Manual Sequence Testing

   Initial State: System with active short-term conversational memory.

   Input/Condition: Testers execute the following conversational sequences:

   1. Summarize my latest unread email. Reply to it with thanks.
   2. Draft an email to John. Change the subject to Project Update.
   3. Show my calendar for tomorrow. Add office hours at 4 PM.
   4. Create a calendar event for Friday at 2 PM. Move it to 3 PM.
   5. Search the web for Hamilton weather. Summarize it.
   6. Search the web for nearby walk-in clinics. Open the first result and summarize it.
   7. Make a Notion page called Drinking Water Tracker. Add today's intake to it.
   8. Make a Notion page called Capstone Tasks. Add testing checklist to it.

9. Summarize the file named Capstone Notes. Read it aloud.
10. Show my latest drafted email. Send it.

Output/Result: At least `FOLLOWUP_SUCCESS_MIN` of the sequences are completed with correct interpretation of follow-up references such as "it", "that", or omitted repeated context.

How test will be performed: Each sequence will be entered by a tester. The interpreted actions and resulting displayed outputs will be reviewed to confirm that contextual references are resolved correctly.

Functional Requirements Covered: FUNC.R.6

2. **Test FUNC.R.7 – Interaction and Task Logging Verification**

   Type: Functional / Inspection

   Initial State: Logging module enabled with write access to the configured storage location.

   Input/Condition: The following sessions are performed, with a mix of successful and failed tasks using both voice and text: summarize email, draft email, create calendar event, make Notion page, browser search, file summary, invalid command, send draft then cancel, browser request with network disabled, and undo last action.

   Output/Result: Each session generates a timestamped log entry containing the received input, interpreted intent, resulting action, and final outcome.

   How test will be performed: After each session, reviewers will inspect the recorded logs for completeness, correct timestamps, session identifiers, action summaries, and outcome fields.

   Functional Requirements Covered: FUNC.R.7

3.1.5 Usability and Safety

1. **Test FUNC.R.8 – High-Level Command Usability**

   Type: Usability / Observational Testing

   Initial State: Deployed system interface with voice and text interaction enabled.

Input/Condition: `USABILITY_PARTICIPANTS` participants attempt the following core tasks using Proxi's high-level interface: summarize an unread email, draft an email, show tomorrow's calendar, create a calendar event, search the web, summarize a browser result, make a Notion page, update a Notion page, summarize a file, and use the help command.

Output/Result: At least `HIGHLEVEL_TASK_MIN` of attempted tasks are completed successfully without requiring users to interact with low-level system details such as raw file paths, terminal commands, or manual system configuration steps.

How test will be performed: Observers will record whether each task is completed successfully, whether low-level system details were needed, and where users hesitated or failed. Completion rate will be computed over the full task set across all participants.

Functional Requirements Covered: FUNC.R.8

2. **Test FUNC.R.9 – Privileged Action Confirmation**

   Type: Functional / Safety Validation

   Initial State: System running with permission-sensitive actions enabled.

   Input/Condition: The following privileged commands are issued by both voice and text: send the drafted email, delete an email draft, delete a calendar event, remove a Notion page, and clear saved session history. For each command, one trial confirms the action and one trial cancels the action.

   Output/Result: Every privileged command triggers a confirmation prompt before execution. Confirmed actions execute only after explicit approval, and cancelled actions do not execute.

   How test will be performed: Testers will issue each privileged command twice, once followed by confirmation and once followed by cancel. Observers will verify that the prompt appears before execution and that the final result matches the approval or cancellation path.

   Functional Requirements Covered: FUNC.R.9

3.2 Tests for Non-Functional Requirements

This section defines specific tests used to determine whether Proxi meets its non-functional requirements. Each test is derived from the SRS and states how the result will be verified.

3.2.1 Look and Feel Requirements

1. **Test NF.LF.1 – UI Conformance and Readability**

   Type: Static Inspection / Manual Testing

   Initial State: Latest UI build available in the chat-style frontend.

   Input/Condition: Review the main screens, transcript area, status indicators, and primary controls using the following checklist: listen control visible, stop or cancel control visible, undo control visible, status indicator text readable, transcript text readable, minimal visual clutter in the main path, advanced options hidden from the default flow, large click targets, and theme toggle present.

   Output/Result: All checklist items are present and usable in the main workflow.

   How test will be performed: Two reviewers will inspect the UI independently, complete the checklist, compare results, and record any issues with screenshots.

   Non-Functional Requirements Covered: APP.1, APP.2, APP.3, APP.4, APP.5

2. **Test NF.LF.2 – Style and Iconography Check**

   Type: Static Inspection / Manual Testing

   Initial State: Build with spoken output, on-screen text, and the final icon set enabled.

   Input/Condition: Trigger the following outputs and review the resulting labels, captions, and icons: welcome message, task success message, task failure message, confirmation prompt, cancel action, undo action, help response, and one browser summary response.

   Output/Result: Labels are short and plain, spoken output also appears on screen, and icons are consistent and paired with short text labels.

How test will be performed: Reviewers will inspect the displayed strings and visual elements, then record any inconsistencies with screenshots or screen recordings.

Non-Functional Requirements Covered: STY.1, STY.2, STY.3

3.2.2 Usability and Humanity Requirements

1. **Test NF.US.1 – Task-Based Usability and Learnability Study**

   Type: Dynamic / Manual Usability Testing

   Initial State: New users receive a `ORIENTATION_TIME` orientation to Proxi.

   Input/Condition: Each participant performs the following core tasks: summarize an unread email, draft an email, show tomorrow's calendar, create a calendar event, search the web, summarize a browser result, make a Notion page, update a Notion page, summarize a file, and use the "What can I say?" help feature.

   Output/Result: Main actions are discoverable, most everyday tasks are completed without assistance after the orientation, repeated tasks become faster, example commands are easy to find, messages are clear, risky actions are confirmed, and error messages suggest a next step. Survey targets are interpreted using the acceptance criteria defined in Appendix A.

   How test will be performed: Observers will measure task completion, task time, repeated-task improvement, and requests for help. Participants will also complete the usability survey in Appendix A. If a survey target is missed, the team will log the issue, identify the affected workflow, and revise that part of the interface or system response before final delivery.

   Non-Functional Requirements Covered: EOU.R.1, EOU.R.2, EOU.R.3, LEA.R.1, LEA.R.2, UAP.R.1, UAP.R.2, UAP.R.3

2. **Test NF.US.2 – Accessibility, Personalization, and Locale**

   Type: Dynamic Testing / Accessibility Inspection

   Initial State: Build with voice and text enabled; captions, scalable text, and contrast-compliant themes implemented.

Input/Condition: Execute the following voice-only tasks: start setup, ask for help, summarize an unread email, show tomorrow's calendar, create a calendar event, cancel the current action, undo the last action, and exit the app. Then inspect the interface for the following accessibility checks: captions shown for spoken responses, text resizes to `TEXT_RESIZE_REQ` without loss of information, and color contrast meets WCAG 2.2 Success Criterion 1.4.3. Also verify that interface structure and controls satisfy WCAG 2.2 Success Criteria 1.3.1 and 4.1.2 where applicable, and that date, time, and language outputs follow Canadian English formatting.

Output/Result: All listed tasks can be completed through the voice-only path, captions are present for spoken responses, text scaling works without loss of function, contrast is sufficient, interface semantics remain understandable to assistive technologies, and Canadian English formats are used consistently.

How test will be performed: Reviewers will run the voice-only task suite, inspect captions and scaled layouts, check contrast using an accessibility checker, inspect control labels and roles in the UI, and verify locale-formatted outputs in the interface.

Non-Functional Requirements Covered: ACC.R.1, ACC.R.2, PER.R.1, PER.R.2

3.2.3 Performance and Reliability Requirements

1. **Test NF.PF.1 – Latency and Recognition Accuracy Study**

   Type: Dynamic / Automated Performance Testing

   Initial State: Standard hardware environment with typical system load.

   Input/Condition: Run the following commands in a quiet room and then repeat the same commands in a moderately noisy indoor environment: summarize an unread email, draft an email, show tomorrow's calendar, create a calendar event, search the web for Hamilton weather, summarize the browser result, search for nearby pharmacies, make a Notion page, summarize a file, and use the help command.

   Output/Result: Voice responses begin within the SAL.R.1 limit, common actions complete within the SAL.R.2 limit, quiet-environment

recognition accuracy meets POA.R.1, and moderate-noise accuracy meets POA.R.2.

How test will be performed: The team will time each command, compare recognized inputs against expected inputs, compute command accuracy for both environments, and record the results in a test log.

Non-Functional Requirements Covered: SAL.R.1, SAL.R.2, POA.R.1, POA.R.2

2. **Test NF.PF.2 – Robustness and Capacity Study**

   Type: Dynamic / Automated and Manual Testing

   Initial State: System running on standard hardware.

   Input/Condition: Submit the following invalid or adverse inputs: empty command, incomplete command, unsupported command, malformed text input, interrupted speech input, network-dependent request with network disabled, and a cancelled action. Then run the system continuously for at least `SESSION_DURATION_MIN` under normal usage.

   Output/Result: Input errors are logged and reported without a crash, the system continues running after invalid or incomplete commands, continuous operation is maintained for at least `SESSION_DURATION_MIN`, and CPU usage stays within the CAP.R.2 limit during normal operation.

   How test will be performed: Reviewers will trigger each listed failure condition, inspect logs and on-screen errors, monitor whether the system remains usable, and record runtime and CPU usage during the continuous run.

   Non-Functional Requirements Covered: ROFT.R.1, ROFT.R.2, CAP.R.1, CAP.R.2

3. **Test NF.PF.3 – Safety, Extensibility, and Longevity Study**

   Type: Dynamic / Manual and Checklist-Based Testing

   Initial State: System running with undo support, update support, and MCP tool registry enabled.

   Input/Condition: Perform the following actions: delete a draft and cancel, delete a draft and confirm, remove a calendar event and undo

it, add one new MCP tool through the registry, update the build, and simulate an update rollback.

Output/Result: Destructive actions require confirmation, critical actions can be undone in one step, a new feature becomes available without major changes to existing functionality, user settings persist after update, and rollback completes within the LON.R.2 limit.

How test will be performed: Reviewers will execute the listed actions, verify confirmation and undo behaviour, register one new MCP tool, confirm that it loads correctly, update the system, and then measure the rollback process.

Non-Functional Requirements Covered: SAF.R.1, SAF.R.2, SOE.R.1, SOE.R.2, LON.R.1, LON.R.2

3.2.4 Operational and Environmental Requirements

1. **Test NF.OP.1 – Environment and Ecosystem Fit**

   Type: Dynamic / Checklist-Based Testing

   Initial State: Fresh install on current Windows and macOS systems with required integrations enabled.

   Input/Condition: Run the system indoors under normal operating conditions, including moderate indoor noise, user distance within `MIC_DISTANCE_MAX` of the microphone, and room temperature between `ROOM_TEMP_MIN` and `ROOM_TEMP_MAX`. Perform a browser search, email task, file summary task, and calendar task while other common desktop applications are also running.

   Output/Result: The system operates effectively within the expected physical environment, speech recognition remains usable within the specified distance and indoor noise conditions, and automation does not interfere with other running applications.

   How test will be performed: Reviewers will execute the listed tasks under the specified conditions, note any environmental failures, and check whether other applications continue to function normally.

   Non-Functional Requirements Covered: EPE.R.1, EPE.R.2, EPE.R.3, EPE.R.4, WER.R.1, WER.R.2

2. **Test NF.OP.2 – Interfaces, Packaging, and Release Check**

   Type: Dynamic / Inspection and Checklist Testing

   Initial State: Release candidate build, release notes, and deployment package available.

   Input/Condition: Perform a clean install, verify external integrations with email, browser, file, and speech services, inspect communication protocol use, inspect external interaction logs, and review the packaged release.

   Output/Result: Interfaces with adjacent systems use the expected protocols and logging, the product installs with minimal setup, the release follows the versioning scheme, and the release package is version controlled and archived.

   How test will be performed: Reviewers will perform a clean install, execute one task per integration, inspect logs for external interactions, and review release artifacts, tags, and packaged files.

   Non-Functional Requirements Covered: IAS.R.1, IAS.R.2, IAS.R.3, IAS.R.4, PRD.R.1, PRD.R.2, PRD.R.3, REL.R.1, REL.R.2

3.2.5 Maintainability and Support Requirements

1. **Test NF.MS.1 – Onboarding, Support, and Adaptability**

   Type: Dynamic / Checklist-Based Testing

   Initial State: Fresh clone of the repository, README present, Report Problem feature present, and configuration file editable.

   Input/Condition: A new contributor follows the README to build and run the project, adds one new MCP tool using a module and registry entry, generates a support bundle through Report Problem, and changes the default app configuration without changing code.

   Output/Result: The build and run process succeeds without help, a new MCP tool is added without modifying existing tools, the support bundle contains recent logs and basic system information with consent, and the system continues working after only a configuration change.

   How test will be performed: The team will time the setup, verify that the new contributor can complete the README steps, add and

register one tool, inspect the generated ZIP bundle, modify the default app configuration, and rerun the affected task flow.

Non-Functional Requirements Covered: MTN.R.1, MTN.R.2, SUP.R.1, SUP.R.2, ADP.R.1, ADP.R.2

3.2.6 Security Requirements

1. **Test NF.SEC.1 – Access, Integrity, Privacy, and Immunity**

   Type: Dynamic / Inspection and Security Scanning

   Initial State: Fresh install, privacy policy available, and dependency scanner configured.

   Input/Condition: Verify that no account is required to use the app, trigger actions that require explicit permission, inspect stored and transmitted data for protection and minimization, review user consent steps for third-party services, and run dependency vulnerability scans.

   Output/Result: No registration is required, required permissions must be explicitly granted by the user, stored or transmitted data is protected from unauthorized alteration, stored data handling aligns with PIPEDA expectations, third-party data sharing is minimized and consented to, and no known dependency vulnerabilities remain unresolved at release time.

   How test will be performed: Reviewers will execute permission flows, inspect privacy-related behaviour and consent prompts, inspect logs and network traffic at a high level, and run software composition and dependency scans.

   Non-Functional Requirements Covered: ACS.R.1, ACS.R.2, INT.R.1, PRIV.R.1, IMM.R.1, IMM.R.2

2. **Test NF.SEC.2 – Replay or Unauthorized Voice Command**

   Type: Dynamic / Manual Security Testing

   Initial State: System running with voice input enabled.

   Input/Condition: Play a recorded high-risk voice command, or have a different person issue a destructive command such as deleting a draft or removing an event.

Output/Result: The system does not execute the action silently. It must either reject the command, require explicit confirmation, or log the event for review.

How test will be performed: Reviewers will issue a risky command using a recording and then using a second speaker, and will verify that the action is not completed without the expected safeguard.

Non-Functional Requirements Covered: SAF.R.1, ACS.R.2

3.2.7 Cultural Requirements

1. **Test NF.CUL.1 – Language and Tone Check**

   Type: Static Inspection / Manual Testing

   Initial State: Final strings, prompts, and language settings available.

   Input/Condition: Review common screens and messages, including welcome text, help text, error messages, confirmation prompts, task success messages, and cancellation messages. Then switch languages and repeat a subset of basic flows.

   Output/Result: Wording remains polite and culturally neutral, avoids slang and confusing references, and multiple languages support the basic interaction flow.

   How test will be performed: Two reviewers will inspect the default language strings, then one reviewer will switch languages, repeat the selected flows, and record any broken or untranslated text.

   Non-Functional Requirements Covered: CULR.R.1, CULR.R.2

3.2.8 Compliance Requirements

1. **Test NF.COM.1 – Legal and Standards Compliance Check**

   Type: Static Inspection / Checklist Testing

   Initial State: License file, privacy policy, accessibility notes, and release artifacts available in the repository.

   Input/Condition: Review legal and standards compliance against the following checklist: PIPEDA handling for stored user data, MIT license presence and wording, WCAG 2.2 accessibility evidence, and

documentation of software quality and security practices aligned with ISO 25010 and ISO 27001 goals.

Output/Result: Required legal notices are present, the release uses the MIT license correctly, privacy documentation aligns with stated behaviour, and standards-compliance evidence is present and consistent with the implementation.

How test will be performed: Reviewers will inspect policies, license files, release notes, and accessibility evidence, then record any missing or inconsistent items in a compliance checklist.

Non-Functional Requirements Covered: LGL.R.1, LGL.R.2, STDCOMP.R.1, STDCOMP.R.2

3.3 Traceability Between Test Cases and Requirements

Tables 2, 3, and 4 show that every requirement defined in the SRS is covered by at least one planned system test.

Table 2: Functional Requirements Traceability Matrix

Test ID	F1	F2	F3	F4	F5	F6	F7	F8	F9
FUNC.R.1	×								
FUNC.R.2		×							
FUNC.R.3			×						
FUNC.R.4				×					
FUNC.R.5					×				
FUNC.R.6						×			
FUNC.R.7							×		
FUNC.R.8								×	
FUNC.R.9									×

Functional requirement keys: F1 = FUNC.R.1, F2 = FUNC.R.2, F3 = FUNC.R.3, F4 = FUNC.R.4, F5 = FUNC.R.5, F6 = FUNC.R.6, F7 = FUNC.R.7, F8 = FUNC.R.8, F9 = FUNC.R.9.

Table 3: Non-Functional Requirements Traceability Matrix Part 1

Req ID	LF1	LF2	US1	US2	PF1	PF2
APP.1	×					
APP.2	×					
APP.3	×					
APP.4	×					
APP.5	×					
STY.1		×				
STY.2		×				
STY.3		×				
EOU.R.1			×			
EOU.R.2			×			
EOU.R.3			×			
LEA.R.1			×			
LEA.R.2			×			
UAP.R.1			×			
UAP.R.2			×			
UAP.R.3			×			
ACC.R.1				×		
ACC.R.2				×		
PER.R.1				×		
PER.R.2				×		
SAL.R.1					×	
SAL.R.2					×	
POA.R.1					×	
POA.R.2					×	
ROFT.R.1						×
ROFT.R.2						×
CAP.R.1						×
CAP.R.2						×

Test keys for Table 3: LF1 = NF.LF.1, LF2 = NF.LF.2, US1 = NF.US.1, US2 = NF.US.2, PF1 = NF.PF.1, PF2 = NF.PF.2.

Table 4: Non-Functional Requirements Traceability Matrix Part 2

Req ID	PF3	OP1	OP2	SEC1	SEC2	COM1
SAF.R.1	×				×	
SAF.R.2	×					
SOE.R.1	×					
SOE.R.2	×					
LON.R.1	×					
LON.R.2	×					
EPE.R.1		×				
EPE.R.2		×				
EPE.R.3		×				
EPE.R.4		×				
WER.R.1		×				
WER.R.2		×				
IAS.R.1			×			
IAS.R.2			×			
IAS.R.3			×			
IAS.R.4			×			
PRD.R.1			×			
PRD.R.2			×			
PRD.R.3			×			
REL.R.1			×			
REL.R.2			×			
ACS.R.1				×		
ACS.R.2				×	×	
INT.R.1				×		
PRIV.R.1				×		
IMM.R.1				×		
IMM.R.2				×		
LGL.R.1						×
LGL.R.2						×
STDCOMP.R.1						×
STDCOMP.R.2						×

Test keys for Table 4: PF3 = NF.PF.3, OP1 = NF.OP.1, OP2 = NF.OP.2, SEC1 = NF.SEC.1, SEC2 = NF.SEC.2, COM1 = NF.COM.1.

4 Unit Test Description

This section describes the planned unit tests for Proxi after the MIS was completed. The goal of these tests is to check important module logic before moving to integration and system testing.

Our general testing approach is simple. For each selected module, we test the normal case first, then important edge cases and error cases. Most of these tests will be automated. For modules that depend on outside services or tools, mocks will be used instead of the real service.

To keep the unit test section readable, each planned test is given a short unit-test ID for traceability and a descriptive test name for implementation.

4.1 Unit Testing Scope

The main modules in scope for direct unit testing are BH-Input, BH-NLU, BH-Plan, BH-Safety, BH-Feedback, and BH-UI.

These modules were chosen because they contain important logic, are easy to test in isolation, and have clear access programs in the MIS.

The following are outside direct unit test scope: external speech services, external MCP tools, browser, email, calendar, Notion, and operating-system behaviour. These will be verified through mocks, integration tests, and system tests instead.

Some support modules, such as SD-Types, SD-Store, SD-AIClient, SD-ToolRegistry, and SD-Log, are lower priority for direct unit testing in this document. They are either simple support modules or are tested indirectly through higher-priority modules.

4.2 Tests for Functional Requirements

Most unit verification will be done using automated tests. The test cases below are grouped by module and use short unit-test IDs so they can be traced later in the report.

4.2.1 Tests for BH-Input

BH-Input is unit tested because it handles voice and text input, state changes, and input retrieval.

1. **UT.INPUT.1** – `test_bh_input_init_sets_idle_state`

   Type: Functional, Dynamic, Automatic

   Initial State: Module not initialized.

   Input/Condition: Call `initInput()`.

   Output/Result: State becomes Idle, mode becomes TextOnly, and stored text is empty.

   How test will be performed: An automated unit test will call `initInput()` and check the returned or stored state.

2. **UT.INPUT.2** – `test_bh_input_start_capture_from_idle`

   Type: Functional, Dynamic, Automatic

   Initial State: Module initialized and idle.

   Input/Condition: Call `startCapture(Mixed)`.

   Output/Result: State becomes Listening and mode becomes Mixed.

   How test will be performed: An automated unit test will start capture and check the new state.

3. **UT.INPUT.3** – `test_bh_input_duplicate_start_raises_error`

   Type: Functional, Dynamic, Automatic

   Initial State: Module already listening.

   Input/Condition: Call `startCapture(VoiceOnly)` again.

   Output/Result: Exception `AlreadyCapturing` is raised.

   How test will be performed: An automated unit test will start capture once and then call it again.

4. **UT.INPUT.4** – `test_bh_input_stop_capture_returns_idle`

   Type: Functional, Dynamic, Automatic

   Initial State: Module listening or processing.

   Input/Condition: Call `stopCapture()`.

   Output/Result: State returns to Idle.

   How test will be performed: An automated unit test will start capture, stop it, and verify the resulting state.

5. **UT.INPUT.5** – `test_bh_input_get_last_text_raises_when_empty`

   Type: Functional, Dynamic, Automatic

   Initial State: Module initialized with no stored text.

   Input/Condition: Call `getLastText()`.

   Output/Result: Exception `NoInputAvailable` is raised.

   How test will be performed: An automated unit test will initialize the module and request the last text right away.

4.2.2 Tests for BH-NLU

BH-NLU is unit tested because it converts input text into intents and parameters, and its behaviour is mostly deterministic.

1. **UT.NLU.1** – `test_bh_nlu_parse_email_command`

   Type: Functional, Dynamic, Automatic

   Initial State: Parser available.

   Input/Condition: Call `parseText("Draft an email to John about tomorrow's meeting")`.

   Output/Result: Returned intent matches draft-email and includes useful parameters.

   How test will be performed: An automated unit test will compare the returned intent type and fields to the expected values.

2. **UT.NLU.2** – `test_bh_nlu_parse_calendar_command`

   Type: Functional, Dynamic, Automatic

   Initial State: Parser available.

   Input/Condition: Call `parseText("Create a calendar event for Friday at 2 PM called Team Sync")`.

   Output/Result: Returned intent matches create-calendar-event and includes time and title information.

   How test will be performed: An automated unit test will verify the returned type and parsed fields.

3. **UT.NLU.3** – `test_bh_nlu_unsupported_command_raises_parse_error`

   Type: Functional, Dynamic, Automatic

   Initial State: Parser available.

   Input/Condition: Call `parseText("Paint my room blue")`.

   Output/Result: Exception `ParseError` is raised.

   How test will be performed: An automated unit test will give an unsupported command and check the exception.

4.2.3 Tests for BH-Plan

BH-Plan is unit tested because it creates plans from intents and controls execution behaviour. External tools are mocked in these tests.

1. **UT.PLAN.1** – `test_bh_plan_init_sets_default_state`

   Type: Functional, Dynamic, Automatic

   Initial State: Module not initialized.

   Input/Condition: Call `initPlan()`.

   Output/Result: Current plan is null and last status is Pending.

   How test will be performed: An automated unit test will initialize the module and check the stored values.

2. **UT.PLAN.2** – `test_bh_plan_create_plan_for_supported_intent`

   Type: Functional, Dynamic, Automatic

   Initial State: Module initialized with a mocked valid tool registry.

   Input/Condition: Call `planAction(i)` for a supported intent.

   Output/Result: A plan is created with the expected tool and parameters.

   How test will be performed: An automated unit test will mock the tool registry and compare the returned plan.

3. **UT.PLAN.3** – `test_bh_plan_missing_tool_raises_error`

   Type: Functional, Dynamic, Automatic

   Initial State: Module initialized with no matching tool.

Input/Condition: Call `planAction(i)` for an unsupported tool type.

Output/Result: Exception `NoToolFound` is raised.

How test will be performed: An automated unit test will use a mocked registry with no valid match.

4. **UT.PLAN.4** – `test_bh_plan_execute_plan_updates_status`

   Type: Functional, Dynamic, Automatic

   Initial State: Module initialized with a valid mocked plan.

   Input/Condition: Call `executePlan(p)`.

   Output/Result: Status becomes Success or Failed based on the mocked tool result.

   How test will be performed: An automated unit test will mock tool results and check the final status.

5. **UT.PLAN.5** – `test_bh_plan_cancel_without_pending_plan_fails`

   Type: Functional, Dynamic, Automatic

   Initial State: Module initialized with no pending plan.

   Input/Condition: Call `cancelPlan()`.

   Output/Result: Exception `NoPendingPlan` is raised.

   How test will be performed: An automated unit test will call `cancelPlan()` after initialization and verify the exception.

4.2.4 Tests for BH-Safety

BH-Safety is unit tested because it contains important decision rules for risky actions and confirmations.

1. **UT.SAFE.1** – `test_bh_safety_low_risk_action_is_allowed`

   Type: Functional, Dynamic, Automatic

   Initial State: Safety module available.

   Input/Condition: Pass a low-risk action to the safety policy functions.

   Output/Result: The action is automatically allowed.

How test will be performed: An automated unit test will create a mock low-risk action and verify the result.

2. **UT.SAFE.2** – `test_bh_safety_high_irreversible_action_is_blocked`

   Type: Functional, Dynamic, Automatic

   Initial State: Safety module available.

   Input/Condition: Pass a high-risk irreversible action to the safety policy functions.

   Output/Result: The action is blocked.

   How test will be performed: An automated unit test will create a mock high-risk action and verify the result.

3. **UT.SAFE.3** – `test_bh_safety_medium_risk_asks_for_confirmation`

   Type: Functional, Dynamic, Automatic

   Initial State: Safety module available with mocked user response.

   Input/Condition: Pass a medium-risk action to the confirmation path.

   Output/Result: The action requires user confirmation and follows the mocked yes or no response.

   How test will be performed: An automated unit test will mock the user response and verify the returned decision.

4. **UT.SAFE.4** – `test_bh_safety_confirmation_timeout_raises_error`

   Type: Functional, Dynamic, Automatic

   Initial State: Safety module available with no user response.

   Input/Condition: Pass an action that needs confirmation to the confirmation path.

   Output/Result: The module raises the expected timeout error.

   How test will be performed: An automated unit test will simulate no response and verify the timeout behaviour.

4.2.5 Tests for BH-Feedback

BH-Feedback is unit tested because it controls queue behaviour, delivery state, and cancellation logic.

1. **UT.FEED.1** – `test_bh_feedback_init_sets_idle_and_empty_queue`

   Type: Functional, Dynamic, Automatic

   Initial State: Module not initialized.

   Input/Condition: Call `initFeedback()`.

   Output/Result: Queue is empty and status is Idle.

   How test will be performed: An automated unit test will initialize the module and check the queue and status.

2. **UT.FEED.2** – `test_bh_feedback_queue_message_appends_successfully`

   Type: Functional, Dynamic, Automatic

   Initial State: Module initialized.

   Input/Condition: Call `queueMessage("Ready", Both)`.

   Output/Result: The message is added to the queue.

   How test will be performed: An automated unit test will queue one message and check the queue size.

3. **UT.FEED.3** – `test_bh_feedback_speak_now_updates_completed_state`

   Type: Functional, Dynamic, Automatic

   Initial State: Module initialized with mocked output channels.

   Input/Condition: Call `speakNow("Done", Both)`.

   Output/Result: The final status becomes Completed on success.

   How test will be performed: An automated unit test will mock the output channels and check the final status.

4. **UT.FEED.4** – `test_bh_feedback_tts_failure_is_handled`

   Type: Functional, Dynamic, Automatic

   Initial State: Module initialized with a failing mocked TTS path.

   Input/Condition: Call `speakNow("Done", VoiceOnly)`.

Output/Result: The expected failure path or exception is triggered.

How test will be performed: An automated unit test will mock TTS failure and verify the result.

5. **UT.FEED.5** – `test_bh_feedback_cancel_clears_queue`

   Type: Functional, Dynamic, Automatic

   Initial State: Module initialized with queued messages.

   Input/Condition: Call `cancelAll()`.

   Output/Result: Queue is empty and status returns to Idle.

   How test will be performed: An automated unit test will queue messages, cancel them, and check the final state.

4.2.6 Tests for BH-UI

BH-UI is unit tested for state changes and message handling. Visual quality itself is checked mainly by system tests.

1. **UT.UI.1** – `test_bh_ui_init_sets_idle_state`

   Type: Functional, Dynamic, Automatic

   Initial State: UI module not initialized.

   Input/Condition: Call `initUI()`.

   Output/Result: State becomes Idle and last message is empty.

   How test will be performed: An automated unit test will initialize the module and check the state.

2. **UT.UI.2** – `test_bh_ui_update_view_changes_ui_state`

   Type: Functional, Dynamic, Automatic

   Initial State: UI module initialized.

   Input/Condition: Call `updateView(Listening)`.

   Output/Result: UI state changes to Listening.

   How test will be performed: An automated unit test will call `updateView()` and verify the new state.

3. **UT.UI.3** – `test_bh_ui_show_message_stores_message`

   Type: Functional, Dynamic, Automatic

   Initial State: UI module initialized.

   Input/Condition: Call `showMessage("Draft created")`.

   Output/Result: The stored message is updated and the UI state becomes Displaying.

   How test will be performed: An automated unit test will call `showMessage()` and check the resulting state.

4. **UT.UI.4** – `test_bh_ui_prompt_user_returns_response`

   Type: Functional, Dynamic, Automatic

   Initial State: UI module initialized with mocked user response.

   Input/Condition: Call `promptUser("Send this email?")`.

   Output/Result: The prompt path returns the expected user response.

   How test will be performed: An automated unit test will mock the user response and verify the returned value.

5. **UT.UI.5** – `test_bh_ui_clear_screen_resets_display`

   Type: Functional, Dynamic, Automatic

   Initial State: UI module initialized with a shown message.

   Input/Condition: Call `clearScreen()`.

   Output/Result: State returns to Idle and the message is cleared.

   How test will be performed: An automated unit test will show a message, clear the screen, and verify the final state.

4.3 Tests for Non-Functional Requirements

Most non-functional requirements are checked better by integration and system testing. Only a few simple unit-level checks are included here.

4.3.1 Tests for BH-Feedback

1. **UT.NF.FEED.1** – `test_bh_feedback_cancel_leaves_no_queued_messages`

   Type: Non-Functional, Dynamic, Automatic

   Initial State: Feedback module initialized.

   Input/Condition: Queue a few messages and then call `cancelAll()`.

   Output/Result: Queue becomes empty and status returns to Idle.

   How test will be performed: An automated unit test will queue messages, cancel them, and check the final state.

4.3.2 Tests for BH-Safety

1. **UT.NF.SAFE.1** – `test_bh_safety_high_risk_action_is_never_auto_approved`

   Type: Non-Functional, Dynamic, Automatic

   Initial State: Safety module available.

   Input/Condition: Pass a high-risk irreversible action to the safety policy functions.

   Output/Result: The action is blocked or requires confirmation, but it is never auto-approved.

   How test will be performed: An automated unit test will create a mock high-risk action and verify the result.

References

Savinay Chhabra, Gourob Podder, Amanbeer Singh Minhas, and Ajay Singh Grewal. System requirements specification. https://github.com/gkpodder/Capstone/blob/main/docs/DevelopmentPlan/DevelopmentPlan.pdf, 2025a.

Savinay Chhabra, Gourob Podder, Amanbeer Singh Minhas, and Ajay Singh Grewal. System requirements specification. https://github.com/gkpodder/Capstone/blob/main/docs/HazardAnalysis/HazardAnalysis.pdf, 2025b.

Savinay Chhabra, Gourob Podder, Amanbeer Singh Minhas, and Ajay Singh Grewal. System requirements specification. https://github.com/gkpodder/Capstone/blob/main/docs/ProblemStatementAndGoals/ProblemStatement.pdf, 2025c.

Savinay Chhabra, Gourob Podder, Amanbeer Singh Minhas, and Ajay Singh Grewal. System requirements specification. https://github.com/gkpodder/Capstone/blob/main/docs/SRS-Volere/SRS.pdf, 2025d.

Savinay Chhabra, Gourob Podder, Amanbeer Singh Minhas, and Ajay Singh Grewal. System requirements specification. https://github.com/gkpodder/Capstone/tree/main/docs/Extras, 2025e.

Savinay Chhabra, Gourob Podder, Amanbeer Singh Minhas, and Ajay Singh Grewal. User manual. https://github.com/gkpodder/Capstone/tree/main/docs/Extras/UserManual, 2025f.

5 Appendix

This section includes supporting information used by multiple V&V activities.

5.1 Symbolic Parameters

The following symbolic parameters centralize repeated thresholds and test-set sizes used throughout this V&V plan. Using these names instead of repeating raw numeric values improves consistency, traceability, and maintainability.

Table 5: Symbolic Parameters for V&V Activities

Name	Value	Meaning
CMD_SET_SIZE	20	Number of representative commands in the main command acceptance set.
CMD_ACCEPT_MIN	95%	Minimum acceptance rate for valid command input tests.
STT_CMD_COUNT	10	Number of spoken command phrases in the STT dataset.
STT_SPEAKER_COUNT	5	Number of speakers used in the STT sample set.
STT_ACC_MIN	90%	Minimum acceptable speech-to-text word accuracy.
INTENT_SAMPLE_COUNT	100	Number of labeled inputs used in intent recognition testing.
INTENT_ACC_MIN	90%	Minimum acceptable intent recognition accuracy.
TASK_SUCCESS_MIN	85%	Minimum success rate for representative end-to-end task execution.
FEEDBACK_DELAY_MAX	2 s	Maximum delay before visible or spoken feedback appears after completion or known failure.
FOLLOWUP_SUCCESS_MIN	90%	Minimum correct-resolution rate for conversational follow-up sequences.
USABILITY_PARTICIPANTS	10	Target number of participants in the main usability study.
HIGHLEVEL_TASK_MIN	80%	Minimum successful completion rate for high-level tasks without low-level system details.
ORIENTATION_TIME	5 min	Maximum orientation period given before usability testing starts.
TEXT_RESIZE_REQ	200%	Required text resize level

5.2 Usability Survey and Acceptance Criteria

This survey is used as the primary post-session instrument for evaluating perceived usability quality after task-based testing. It is designed to validate the SRS goals for clarity, learnability, responsiveness, accessibility, trust, and acceptance for daily use.

Administration Protocol. The survey is completed immediately after each participant finishes the usability session. Participants are instructed to base ratings on their own observed experience in that session only. Unless otherwise stated, the quantitative questions use a 1–5 Likert scale where 1 indicates the least favorable response and 5 indicates the most favorable response.

1. **How easy was it to complete tasks using voice commands?**
   Response Type: 1–5 Likert (Very Difficult → Very Easy).
   Intent: Assess overall learnability and ease of first-use interaction.

2. **Were the system responses clear and understandable?**
   Response Type: 1–5 Likert (Very Unclear → Very Clear).
   Intent: Measure clarity of feedback, wording quality, and output usefulness.

3. **Did the system react quickly enough to your commands?**
   Response Type: 1–5 Likert (Very Slow → Very Fast).
   Intent: Capture user-perceived responsiveness and latency satisfaction.

4. **How intuitive did the interface feel while completing tasks?**
   Response Type: 1–5 Likert (Very Confusing → Very Intuitive).
   Intent: Evaluate navigation flow, control predictability, and consistency.

5. **Was visual and audio feedback easy to notice and understand?**
   Response Type: 1–5 Likert (Not Accessible → Highly Accessible).
   Intent: Evaluate multimodal feedback accessibility and comprehensibility.

6. **Did you feel confident that Proxi understood your intent?**
   Response Type: 1–5 Likert (Never → Always).
   Intent: Measure trust in recognition and intent interpretation behaviour.

7. **Did confirmations and safety prompts appear at the right time?**
   Response Type: 1–5 Likert (Never Appropriate → Always Appropriate).
   Intent: Evaluate whether high-risk actions are protected without harming flow.

8. **How satisfied are you with the overall experience?**
   Response Type: 1–5 Likert (Very Unsatisfied → Very Satisfied).
   Intent: Measure end-to-end satisfaction and readiness for practical use.

9. **Would you consider using Proxi for daily tasks?**
   Response Type: Yes / No.
   Intent: Estimate adoption potential for routine personal productivity use.

10. **Which task or moment felt most confusing or frustrating?**
    Response Type: Open text.
    Intent: Identify concrete workflow breakdowns and their triggers.

11. **What one change would most improve your experience?**
    Response Type: Open text.
    Intent: Collect high-impact, actionable improvement directions.

Interpretation and Acceptance Rules. Survey responses are analyzed at both item level and theme level.

- Questions 1, 2, 4, 5, 6, and 8 must each achieve a mean score of at least `SURVEY_CLEAR_MIN`.
- Question 3 must also meet `SURVEY_CLEAR_MIN` and be compared against measured timing data from performance tests. If measured latency passes but perception is low, the issue is treated as a feedback and expectation-management problem; if both fail, it is treated as a performance defect.
- Question 7 is interpreted as a safety-usability balance check. Scores below `SURVEY_CLEAR_MIN` trigger targeted review of confirmation frequency, timing, and wording.

- For Question 9, at least `SURVEY_DAILY_USE_MIN` of participants must answer Yes.
- Open-text responses (Questions 10 and 11) are coded into themes such as confusion, latency, recognition error, accessibility barrier, safety prompt friction, and missing guidance, then mapped to affected workflows.
- Any failed numeric threshold or repeated negative theme results in a tracked issue, assigned corrective action, and re-test of the revised workflow before final delivery when feasible.

Appendix - Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Reflection - Team

1. What went well while writing this deliverable?
The strongest part of this deliverable was that each person was able to work from the same testing story even while focusing on different layers of the project. Aman said the planning and validation sections felt most meaningful when they were tied directly back to the SRS, because that kept the document focused on whether Proxi would actually help a user rather than just whether it was technically impressive. Ajay said the non-functional testing work was valuable because it forced the team to define what "works well" should mean for responsiveness, accessibility, and user experience, not just basic correctness. Gourob noted that the functional traceability work became much easier once each requirement had a clearly corresponding test case, which gave the plan a coherent structure. Savinay pointed out that our coordination improved compared with earlier deliverables because responsibilities were communicated earlier, so dependent sections could be written with fewer rewrites. Taken together, the deliverable went well because the team moved beyond isolated writing and started treating the document like a single engineering argument.

2. What pain points did you experience during this deliverable, and how did you resolve them?
The main challenge was balancing ambition with clarity. Aman described the struggle as finding the point where the writing was specific enough to be credible but not so dense that it became hard to follow. Ajay had a similar issue on the non-functional side, where broad ideas such as usability or accessibility had to be translated into testable conditions before they were useful. Gourob emphasized the practical disruption caused by the late submission period and the MSAF process, which created pressure and made the team reorganize quickly once relief had been granted. Savinay added that the team had to recover from a period where several members were unavailable near the deadline, and that uncertainty made planning more difficult than the technical writing itself. The common resolution was a tighter review process: we clarified wording, narrowed vague claims into measurable test cases, and reassigned remaining work so the document could still read as one consistent plan. In practice, that meant using review passes to reduce noise, improve consistency, and keep the final text grounded in evidence.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project?

The team needs a broader mix of testing and evaluation skills than a typical single-layer software project. Aman identified automated testing, coverage-oriented thinking, survey analysis, accessibility evaluation, and evidence organization as especially important, because Proxi will only be convincing if its claims are backed by clear measurements. Ajay highlighted dynamic testing, static testing, automation tools, UI validation, and accessibility-focused inspection, since the interface has to stay usable as the project evolves. Gourob focused on AI-supported evaluation, especially intent recognition and speech input, because those behaviours are core to the system and cannot be judged by intuition alone. Savinay emphasized test planning, scenario design, user-group selection, and the ability to decide whether evidence is strong enough to support a claim, since Proxi depends on several interacting components and requires more than one kind of validation. Across the team, the shared gap is not just technical testing knowledge but the ability to connect observations to engineering claims in a disciplined way.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

The team sees two main ways to build these skills. The first is structured study: course notes, lab material, documentation, and examples from prior testing assignments. The second is practice on the actual project: building test scripts, running trial validation sessions, checking outputs, and adjusting the plan based on what the evidence shows. Aman said he learns best when theory is connected to something concrete, so he will use course material as a reference but focus on building and reviewing real validation artifacts. Ajay preferred small prototype checks and direct UI testing, because practical runs make the tooling and interface tradeoffs easier to understand than reading alone. Gourob planned to rely more heavily on implementation and analysis of real outputs, since he already has a strong technical base and learns best by measuring the system directly. Savinay said he would use both study and examples from open-source projects, but expects the applied examples to be most useful because they show how tests, evidence, and quality checks fit

together in practice. The team's overall choice is to combine both approaches, using theory to set the standard and practice to prove that the standard is met.